Mini Colossal-AI: Distributed Parallelism from Scratch

Phase 3 — Communication-Aware Placement on Heterogeneous GPU Clusters

Dinesh Chand Goutham P Meena Chidambaram Nadhiya R S

MTech-AI, IISc Bangalore

Problem Statement & Motivation

- Large language models (100M–billions of params) **exceed single-GPU memory**
- Distributed training is essential—but **which parallelism strategy** to use?
- Production frameworks (DeepSpeed, Colossal-AI) hide internals—hard to understand trade-offs

Our goal: Build every parallelism strategy *from scratch* using only `torch.distributed` primitives, and study how **hardware topology** affects their relative performance.

Mini Colossal-AI — Four Strategies from Scratch

Data Parallelism (DP): replicate model, sync gradients

ZeRO: shard optimizer states

Tensor Parallelism (TP): split layers, 2 all-reduces/block

Pipeline Parallelism (PP): split stages, send activations

Phase 2 Recap & Phase 3 Questions

Phase 2: 4× g4dn.xlarge, 1 T4/node, **all TCP**

Strategy	tok/s	Mem
Single GPU	1,361	7.67 GB
DP (NCCL)	1,393	7.67 GB
TP (4-way)	959	3.38 GB
PP 1F1B	3,070	2.56 GB
DP2×PP2	2,281	4.08 GB

PP dominated; TP/DP bottlenecked by slow TCP.

Phase 3 Research Questions:

1. How does **PCIe vs. TCP** change strategy ranking?
2. Can **communication-aware placement** unlock 3D hybrid throughput?
3. Do trends hold across **architectures**?

Faculty feedback: "Experiments with more models showing similar trends should be done."

⇒ Add **T5-base** (encoder-decoder) alongside GPT-2 (decoder-only).

Hardware: Phase 2 $\rightarrow$ Phase 3

	Phase 2: g4dn.xlarge	**Phase 3: g4dn.12xlarge**
Nodes	4	2
GPUs / node	1$\times$ T4 (16 GB)	4$\times$ T4 (16 GB each)
Total GPUs	4	8
Intra-node	N/A (1 GPU/node)	PCIe ($\sim$15 GB/s)
Inter-node	TCP ($\sim$0.6 GB/s)	TCP ($\sim$0.6 GB/s)

Phase 3 has a **mixed-speed topology**: fast PCIe within each node, slow TCP between nodes.

25$\times$ bandwidth gap $\Rightarrow$ enables communication-aware placement.

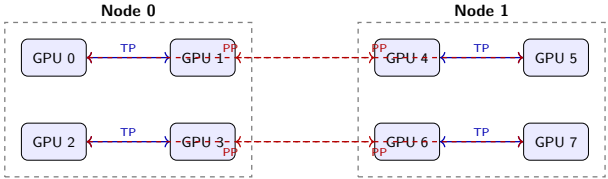
Communication-Aware Placement

Place the **heaviest** axis on the **fastest** link:

Placement	Inter-node	Intra-node
Good	PP	DP + TP
TP inter	TP	DP + PP
Bad	DP	PP + TP

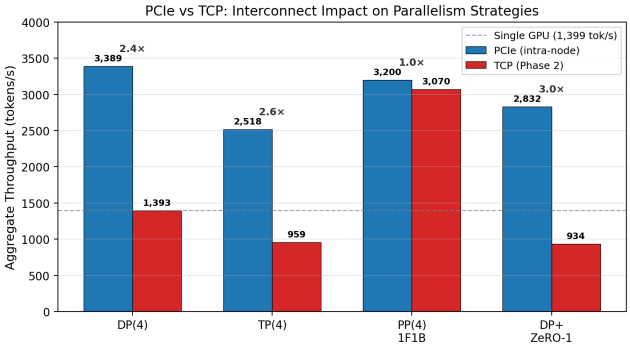
Full 3D mesh: **DP=2** $\times$ **PP=2** $\times$ **TP=2**

No code changes—only reorder mesh axes in the unified 3D plugin.



Blue: TP on PCIe **Red dashed:** PP on TCP

PCIe vs. TCP: Why Placement Matters



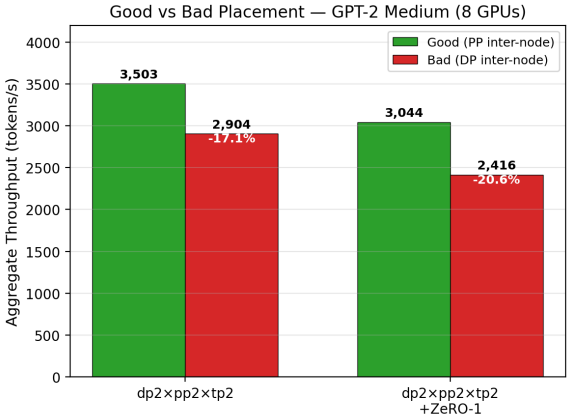
Single-node PCIe vs. Phase 2 TCP:

Strategy	PCIe	vs TCP
DP(4)	3,389	2.4×
TP(4)	2,518	2.6×
PP(4) 1F1B	3,200	1.04×
ZeRO-1	2,832	3.0×

DP and TP **need** fast links (2.4–3×).
PP **tolerates** slow links (1.04×).

⇒ Put PP on TCP, keep DP+TP on PCIe.

Good vs. Bad Placement — GPT-2 Medium (8 GPUs)



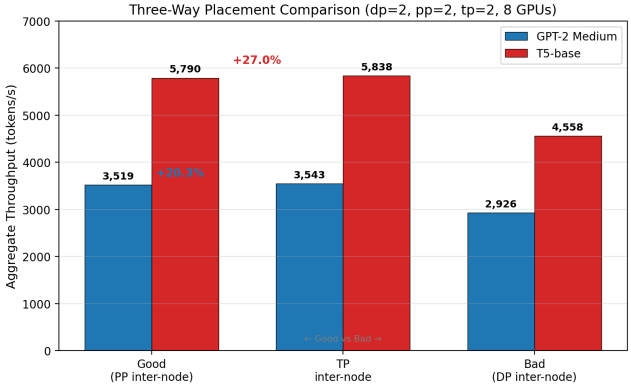
dp=2, pp=2, tp=2:

Config	Good	Bad	Δ
Base	3,503	2,904	−17%
+ZeRO-1	3,044	2,416	−21%

Good placement is ∼**20% faster**.

ZeRO-1 **widens** the gap—extra optimizer-state communication adds to the DP axis load forced onto slow TCP.

Three-Way Placement: A Surprising Finding



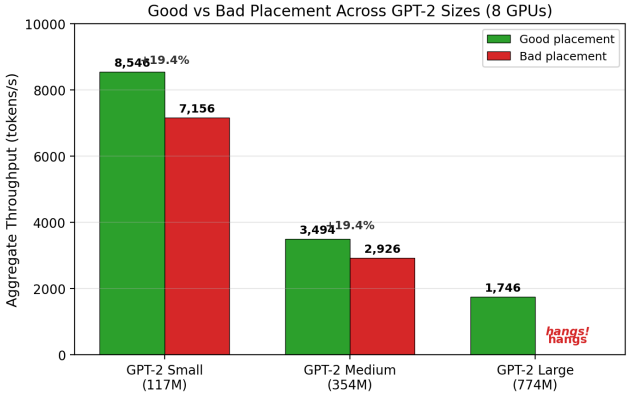
Placement	GPT-2	T5
Good (PP inter)	3,519	5,790
TP inter-node	3,543	5,838
Bad (DP inter)	2,926	4,558
Good vs Bad	+20%	+27%

Surprise: TP inter-node ≈ Good!

At TP=2, messages are small (KB–MB). **DP is the real bottleneck**—it sends the *entire* gradient (∼1.4 GB for GPT-2, ∼950 MB for T5) in one shot.

Correct heuristic: Keep DP on the fastest link.

Multi-Model GPT-2 Placement (8 GPUs)



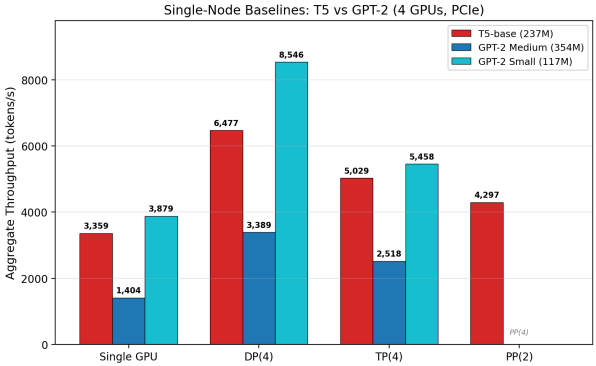
Model	Good	Bad
Small (117M)	8,546	7,156
Medium (354M)	3,494	2,926
Large (774M)	1,746	*hangs*

Good placement wins **consistently** (∼19–20% faster across all sizes).

GPT-2 Large **hangs** under bad placement due to NCCL P2P contention when large activations use shared-memory transport.

Good placement avoids this by routing PP over TCP.

T5-base: Encoder-Decoder Challenges

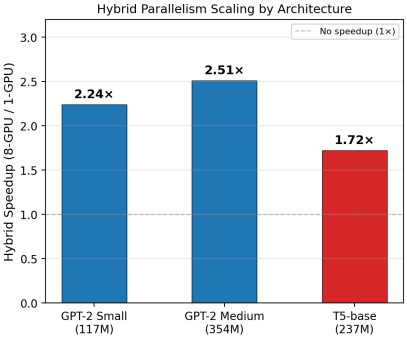


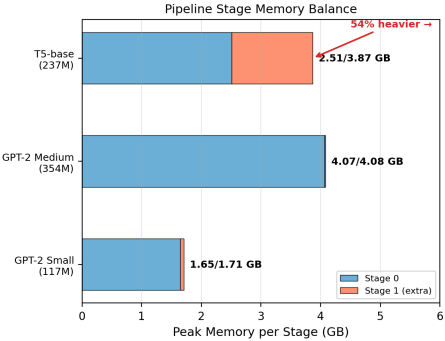
T5-base: 12 enc + 12 dec layers, 237M params

Two key differences from GPT-2:

1. **Pipeline imbalance:**
   - Encoder stage: 2.51 GB
   - Decoder stage: 3.87 GB (**54% heavier**)
   - Encoder idles waiting for decoder
2. **Extra TP communication:**
   - Cross-attention → +1 all-reduce/block
   - T5: **60** all-reduces/fwd pass
   - GPT-2 Med: 48 all-reduces/fwd pass
   - 25% more communication

Architecture Determines Scaling





Model	Params	1-GPU	8-GPU	Speedup	PP Mem (S0/S1)
GPT-2 Small	117M	3,879	8,678	**2.24**×	1.65 / 1.71 GB (balanced)
GPT-2 Medium	354M	1,404	3,519	**2.51**×	4.07 / 4.08 GB (balanced)
T5-base	237M	3,359	5,790	**1.72**×	2.51 / 3.87 GB (**54% imbalance**)

GPT-2 Small (117M) is **smaller** than T5-base (237M) yet scales **better** (2.24× vs 1.72×)
⇒ The gap is **architectural**, not size-related.

Key Findings

1. **Communication-aware placement matters:**
   Keeping DP+TP on the fast intra-node link delivers **17–27% higher throughput** consistently across all models tested.
2. **DP is the most placement-sensitive axis** (not TP):
   DP inter-node: 17–27% penalty — TP inter-node at TP=2: ∼0% penalty.
   DP gradient sync is bandwidth-bound (∼1 GB+); TP messages are small at low degrees.
3. **Placement effect is stronger for T5 (+27%) than GPT-2 (+20%):**
   T5's cross-attention gradients increase the DP synchronization volume.
4. **Architecture determines hybrid parallelism scaling:**
   Symmetric GPT-2: 2.2–2.5× — Asymmetric T5: 1.72×
   Due to pipeline imbalance (54%) and 25% more TP all-reduces from cross-attention.

Conclusion & Future Work

Summary:

- Built **Mini Colossal-AI**—4 parallelism strategies from scratch using `torch.distributed`
- Phase 2 (all-TCP): PP wins, TP/DP bottlenecked by network
- Phase 3 (PCIe + TCP): Communication-aware placement unlocks 3D hybrid parallelism
- Adding T5 reveals hybrid scaling benefits are **not universal**—they depend on architecture
- Validates placement heuristics used by Colossal-AI and Megatron-LM

Future Work:

- Sequence parallelism for long-context models
- Auto-tuning: select mesh dimensions given hardware topology
- Evaluation on NVLink + InfiniBand clusters

Questions?

Dinesh Chand Goutham P Meena Chidambaram Nadhiya R S

MTech-AI, IISc Bangalore